LCS Plagiarism Checker: Dynamic Programming on Text Similarity

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Web Application and Algorithm Implementation

A plagiarism checker was built with **Flask** (Python backend) and HTML/CSS/JavaScript (frontend). The user pastes two documents into side-by-side text areas and presses **"Check Plagiarism"**; the server computes the **Longest Common Subsequence (LCS)** via bottom-up dynamic programming, backtracks through the DP table to recover the LCS string and a character-level diff, and returns the similarity score with a colored diff view.

Step	Description
1. DP fill	`dp[i][j] = dp[i-1][j-1]+1` on match, else `max(dp[i-1][j], dp[i][j-1])`
2. Backtrack	Walk `(m, n)` → `(0, 0)`; take the diagonal on matches, else follow the larger neighbor
3. Score	Similarity = `dp[m][n] / max(m, n)` × `100%`; diff tags each char as match/removed/added

The interface shows three metric cards **(Similarity, LCS Length, LCS String)** and a diff view where matched characters are plain, characters only in A are red with a strike-through, and characters only in B are highlighted in green (Figure 1).

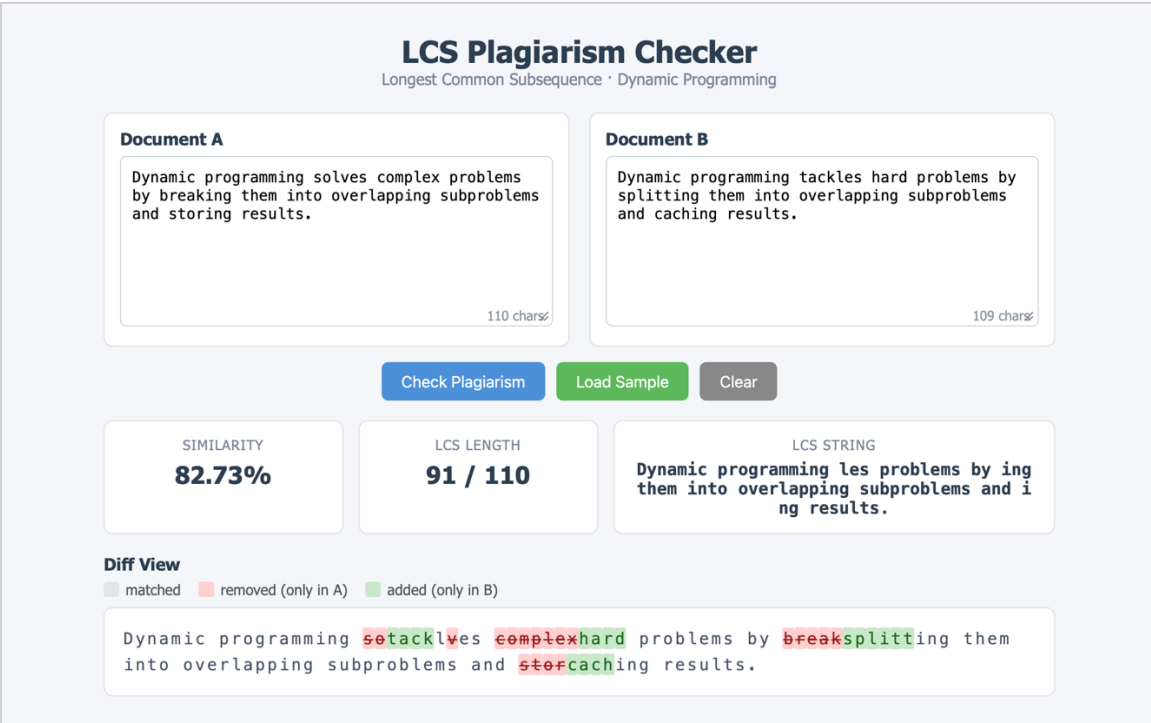

Figure 1. Plagiarism checker comparing two paraphrased sentences (similarity = 82.73%)

2 DP Table Example: “ALGORITHM” vs. “ALTRUISTIC”

The table below shows the full LCS DP matrix for A = “`ALGORITHM`” (m = 9) and B = “`ALTRUISTIC`” (n = 10). Shaded cells mark the characters selected along the backtrack path, yielding the LCS **“ALRIT”** of length 5. The similarity score is `5 / max(9, 10) × 100 = 50.00%`.

	Ø	A	L	T	R	U	I	S	T	I	C
Ø	0	0	0	0	0	0	0	0	0	0	0
A	0	**1**	1	1	1	1	1	1	1	1	1
L	0	1	**2**	2	2	2	2	2	2	2	2
G	0	1	2	2	2	2	2	2	2	2	2
O	0	1	2	2	2	2	2	2	2	2	2
R	0	1	2	2	**3**	3	3	3	3	3	3
I	0	1	2	2	3	3	**4**	4	4	4	4
T	0	1	2	3	3	3	4	4	**5**	5	5
H	0	1	2	3	3	3	4	4	5	5	5
M	0	1	2	3	3	3	4	4	5	5	5

Table 1. LCS DP matrix; shaded cells form the backtrack path producing “ALRIT”

3 Analysis

3.1 Time and Space Complexity

Filling the DP table takes one constant-time update per cell, so the running time is **Θ(mn)**. Storing the full matrix costs **Θ(mn)** space and is required here because the diff view backtracks through it to recover which characters are matched, removed, or added. If only the LCS length were needed, the space drops to **Θ(min(m, n))** by keeping two rolling rows, since `dp[i][j]` depends solely on `dp[i-1][·]` and `dp[i][j-1]`. The quadratic cost is the bottleneck for long inputs: comparing two 100,000-character documents already touches 10^{10} cells, which motivates the heuristics used by production tools.

3.2 Relation to `git diff` and Turnitin

`git diff` is a line-level LCS. Git’s Myers algorithm computes a longest common subsequence of lines and reports the non-matching lines as deletions (−) and insertions (+). Myers refines the plain DP to **O((m+n)*D*)** where *D* is the edit distance — far faster than **Θ(mn)** when the two versions are similar, which is the common case in version control. Our character-level DP and Git’s line-level diff share the same mathematical core; only the “token” differs.

Turnitin cannot afford a full DP against billions of candidate documents. It uses **fingerprinting** (e.g. winnowing of *k*-gram hashes) to shortlist candidates from an index, then applies LCS-style alignment only on the survivors to produce the final similarity report. Our checker is exactly the tool that runs at the final alignment stage; the real production cost lives in the indexing layer that precedes it.